



CS2009

DESIGN AND ANALYSIS OF ALGORITHMS

Dr. Hashim Yasin

**National University of Computer
and Emerging Sciences,
Faisalabad, Pakistan.**

DIVIDE AND
CONQUER



Divide and Conquer

3

- You divide your task (by getting them to distrust each other) and then conquer them piece by piece.
- This is called divide-and-conquer.
- In algorithm design, the idea is to take a problem on a large input,
 - ▣ break the input into smaller pieces,
 - ▣ solve the problem on each of the small pieces,
 - ▣ and then combine the piecewise solutions into a global solution

QUICK SORT



Quick Sort

5

- **Divide:**
 - ▣ Pick any element **p** as the **pivot**, e.g., the first element
 - ▣ Partition the remaining elements into
 - FirstPart**, which contains all elements $< p$
 - SecondPart**, which contains all elements $\geq p$
- **Recursively sort** the **FirstPart** and **SecondPart**
- **Combine:** no work is necessary since sorting is done in place

Quick Sort

6

1. If the number of elements in S is 0 or 1, then return (base case).
2. Pick any element v in S (called the pivot).
3. Partition the elements in S except v into two disjoint groups:
 1. $S_1 = \{x \in S - \{v\} \mid x \leq v\}$
 2. $S_2 = \{x \in S - \{v\} \mid x \geq v\}$
4. Return $\{\text{QuickSort}(S_1) + v + \text{QuickSort}(S_2)\}$

Quick Sort

7

A:

pivot



Partition

FirstPart

SecondPart



Recursive call

Sorted

Sorted

FirstPart

SecondPart



Sorted

Quick Sort

8

```
Quick-Sort(A, left, right)
  if    left  $\geq$  right    return
  else
    middle  $\leftarrow$  Partition(A, left, right)
    Quick-Sort(A, left, middle-1 )
    Quick-Sort(A, middle+1, right)
  end if
```

QUICK SORT

EXAMPLE 1



Quick Sort

5	1	3	8	7	4	6	2
---	---	---	---	---	---	---	---



Elements $<$ pivot



Elements $>$ pivot

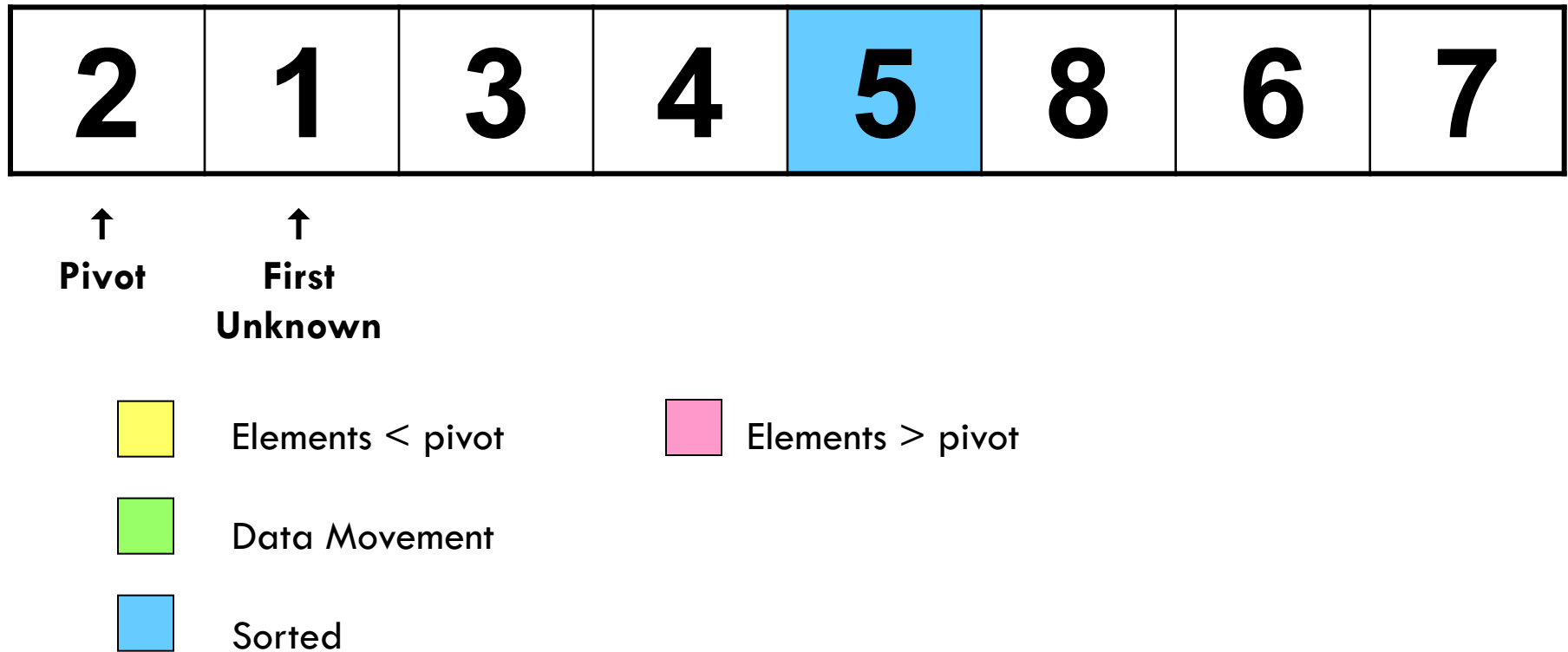


Data Movement

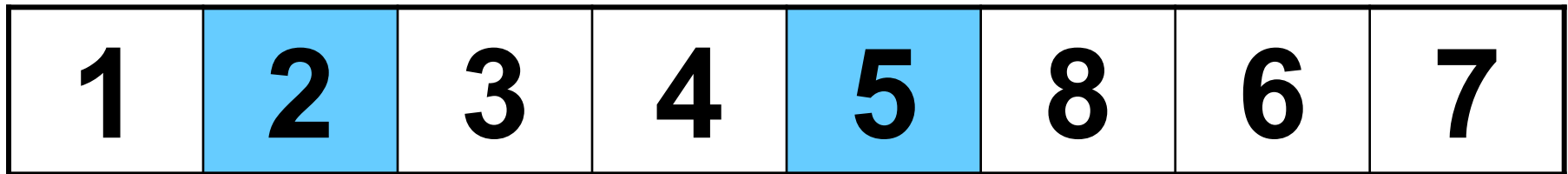


Sorted

Quick Sort



Quick Sort



Elements $<$ pivot



Elements $>$ pivot

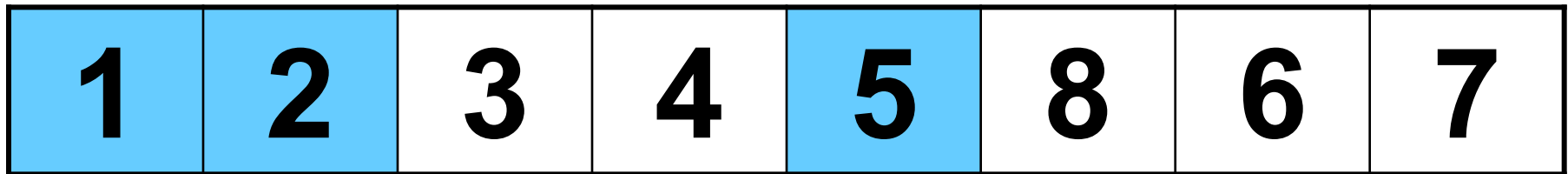


Data Movement



Sorted

Quick Sort



Elements $<$ pivot



Elements $>$ pivot

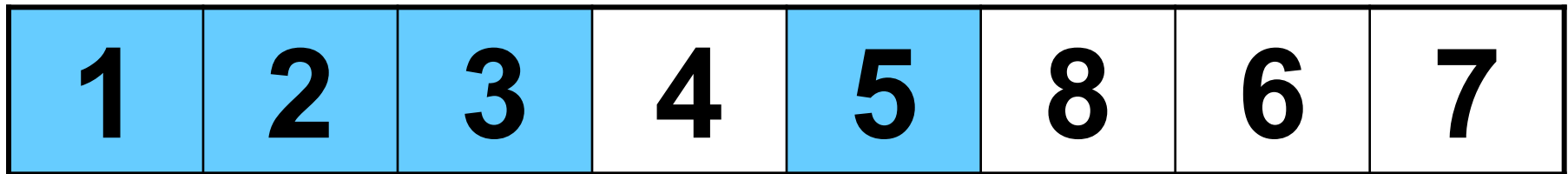


Data Movement



Sorted

Quick Sort



Elements $<$ pivot



Elements $>$ pivot

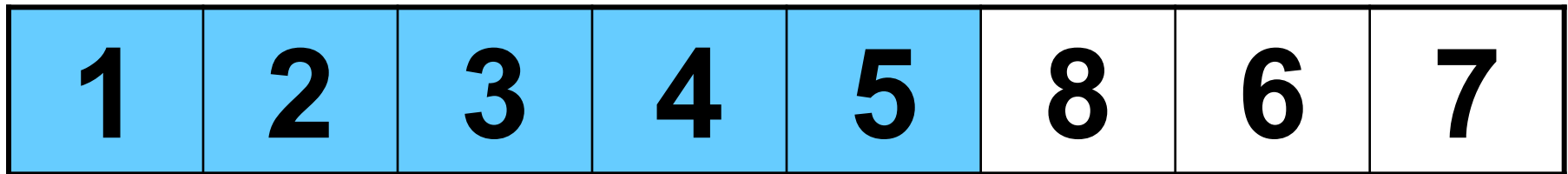


Data Movement



Sorted

Quick Sort



Elements $<$ pivot



Elements $>$ pivot

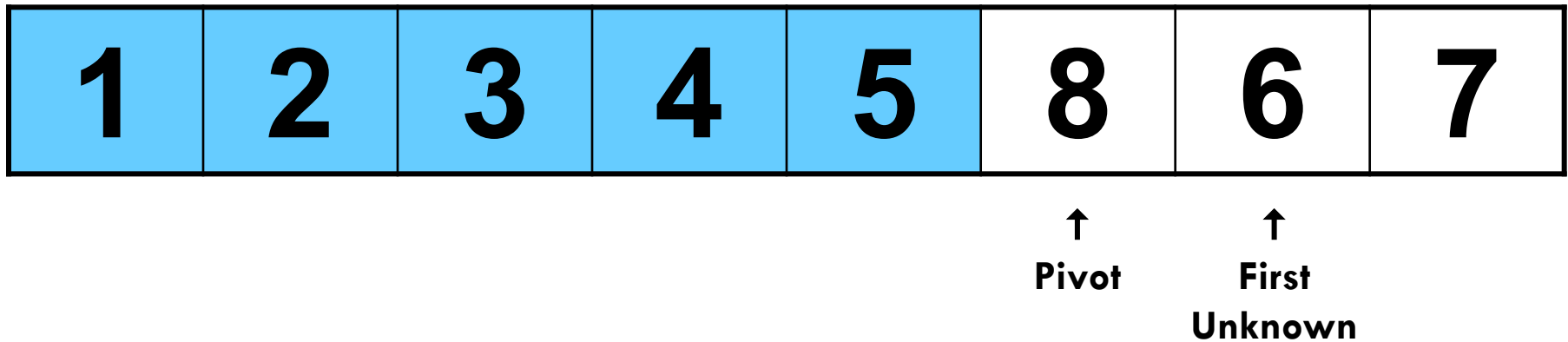


Data Movement



Sorted

Quick Sort



Elements $<$ pivot



Elements $>$ pivot

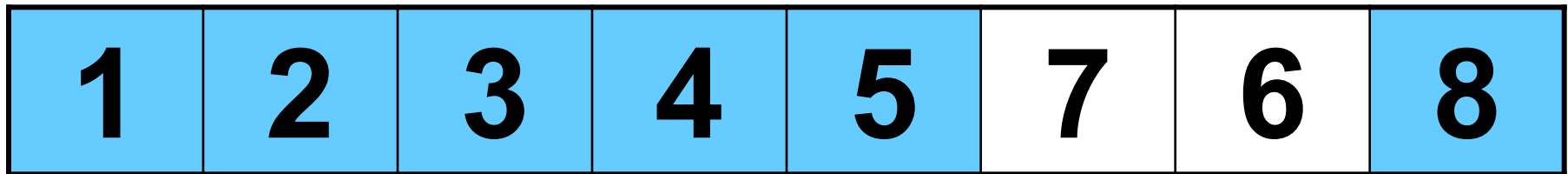


Data Movement



Sorted

Quick Sort



Elements $<$ pivot



Elements $>$ pivot

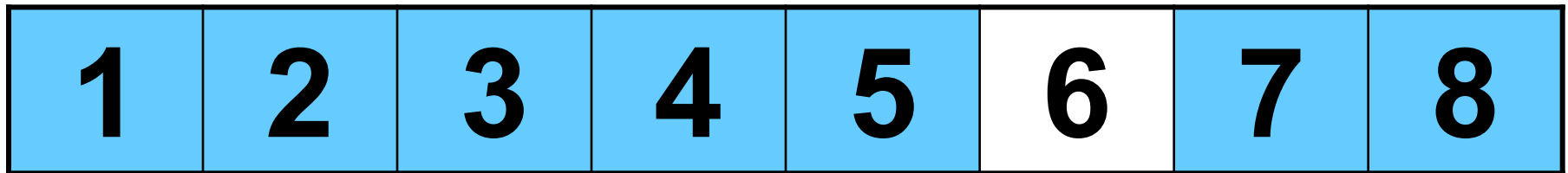


Data Movement



Sorted

Quick Sort



Elements $<$ pivot



Elements $>$ pivot

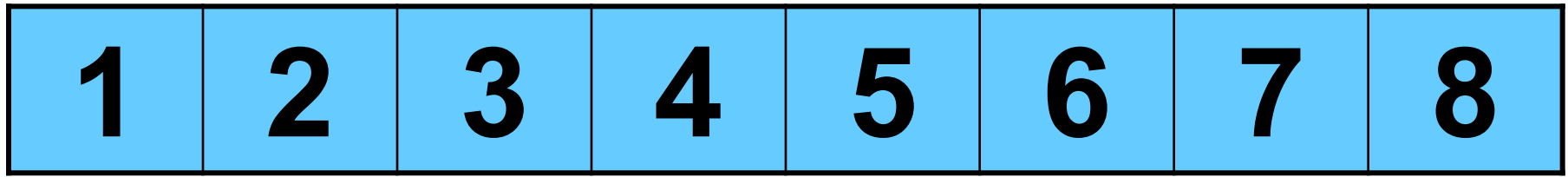


Data Movement



Sorted

Quick Sort



DONE!



Elements $<$ pivot



Elements $>$ pivot



Data Movement



Sorted

Quick Sort ... Issues

20

- How to pick the pivot?
 - ▣ Many methods (discussed later)
- How to partition?
 - ▣ Several methods exist.
 - ▣ We need a method that gives good results, easy to implement and efficient.
 - ▣ We discuss the partition strategy first.

PARTITIONING STRATEGY

EXAMPLE 1



Partition Example

22

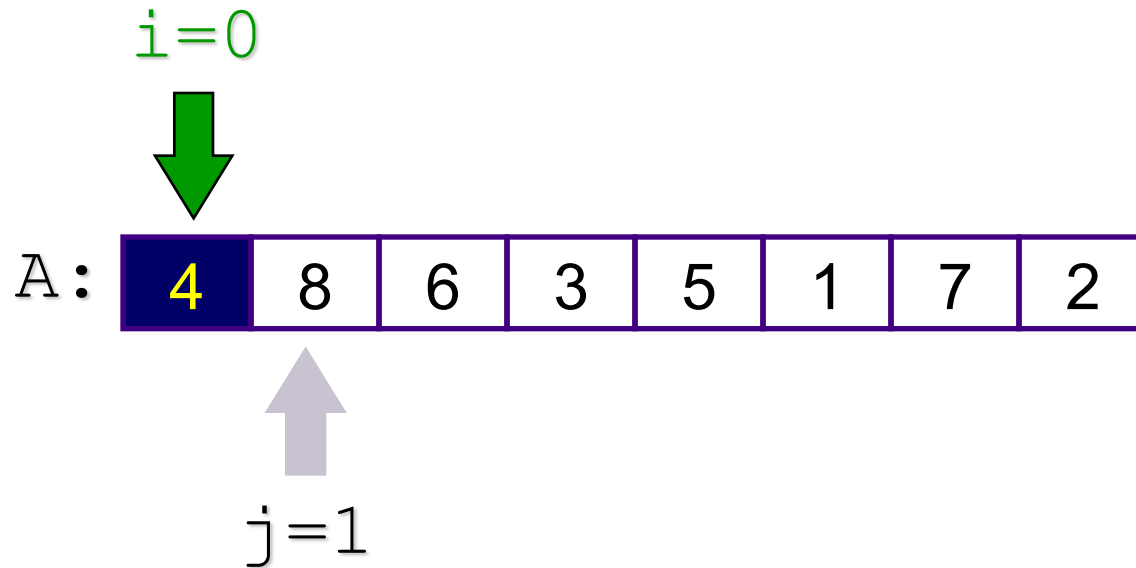
Select First element as Pivot:

A:

4	8	6	3	5	1	7	2
---	---	---	---	---	---	---	---

Partition Example

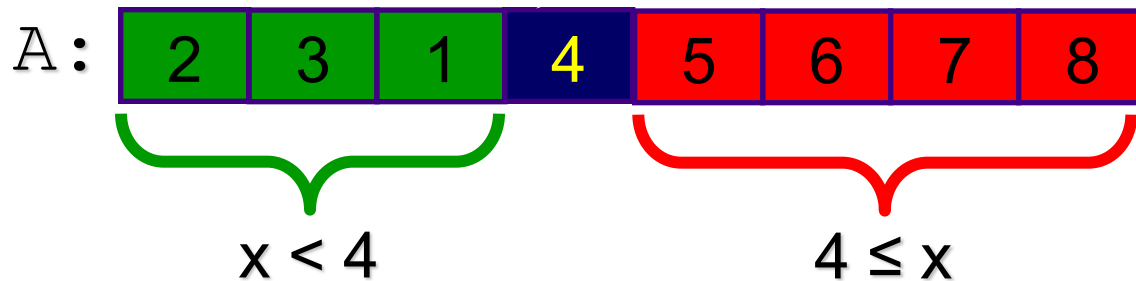
23



Partition Example

24

**pivot in
correct position**



Partition(A, left, right)

```
1.  x ← A[left]
2.  i ← left
3.  for j ← left+1 to right
4.      if A[j] < x then
5.          i ← i + 1
6.          swap(A[i], A[j])
7.      end if
8.  end for j
9.  swap(A[i], A[left])
10. return i
```

$n = \text{right} - \text{left} + 1$

Time: cn for some constant c

PARTITIONING STRATEGY

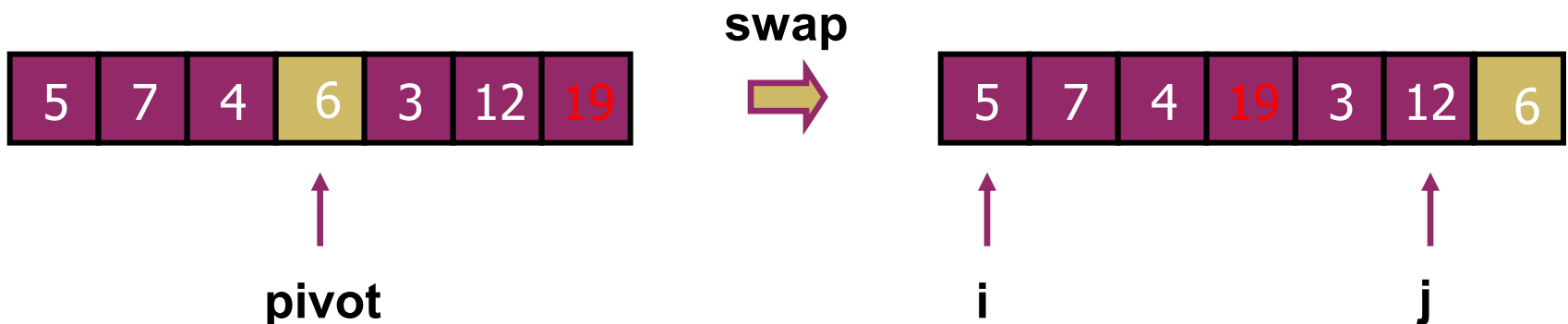
EXAMPLE 2



Partitioning Strategy

27

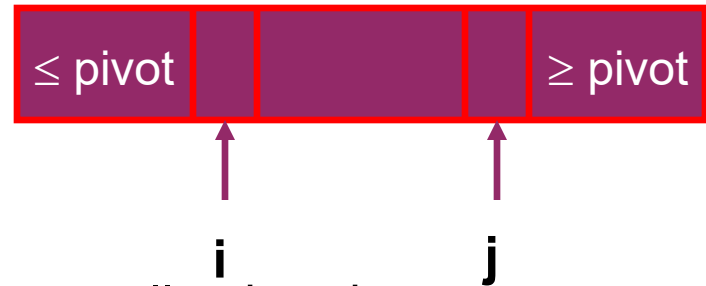
- For now, assume that $\text{pivot} = A[(\text{left} + \text{right})/2]$.
- We want to partition array $A[\text{left} .. \text{right}]$.
- First, get the pivot element out of the way by swapping it with the last element (swap pivot and $A[\text{right}]$).
- Let i start at the first element and j start at the next-to-last element ($i = \text{left}$, $j = \text{right} - 1$)



Partitioning Strategy

28

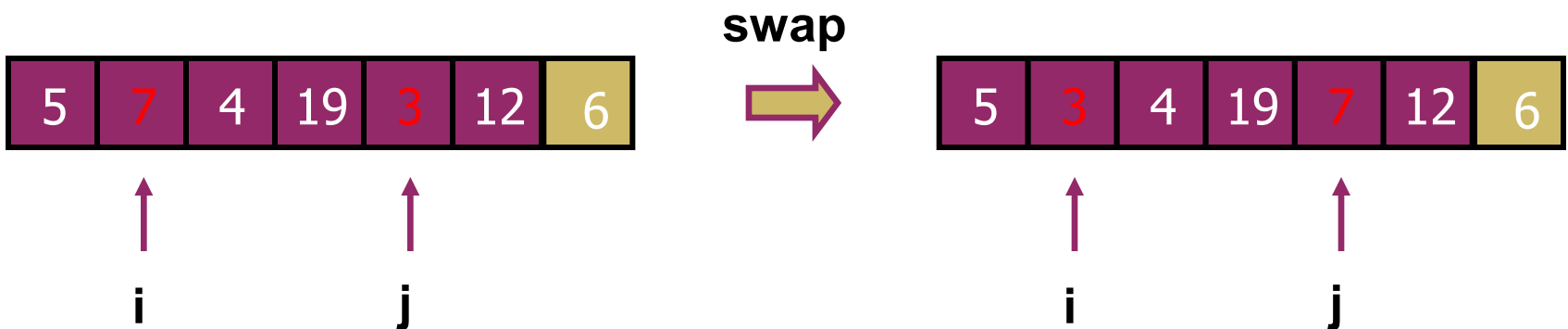
- Want to have
 - ▣ $A[k] \leq \text{pivot}$, for $k < i$
 - ▣ $A[k] \geq \text{pivot}$, for $k > j$
- When $i < j$
 - ▣ Move i right, skipping over elements smaller than the pivot
 - ▣ Move j left, skipping over elements greater than the pivot
 - ▣ When both i and j have stopped
 - $A[i] \geq \text{pivot}$
 - $A[j] \leq \text{pivot} \Rightarrow A[i]$ and $A[j]$ should now be swapped



Partitioning Strategy

29

- When i and j have stopped and i is to the left of j (thus legal)
 - ▣ Swap $A[i]$ and $A[j]$
 - The large element is pushed to the right and the small element is pushed to the left
 - ▣ After swapping
 - $A[i] \leq \text{pivot}$
 - $A[j] \geq \text{pivot}$
 - ▣ Repeat the process until i and j cross



Partitioning Strategy

30

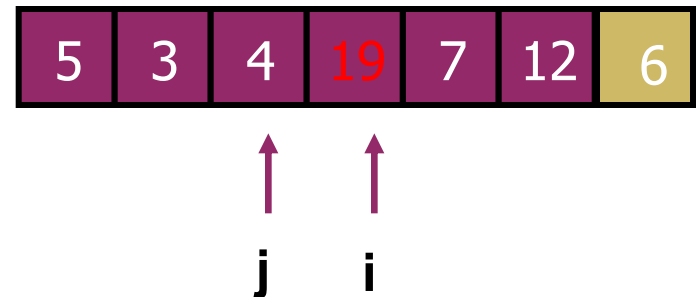
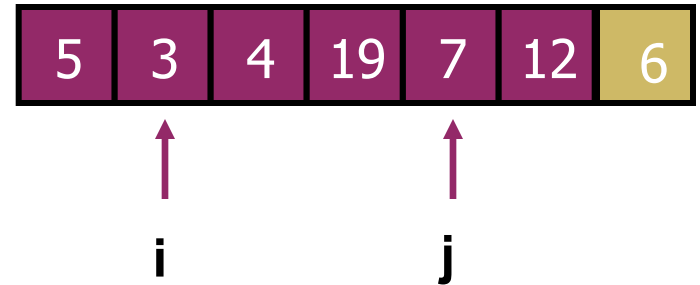
□ When i and j have crossed

□ swap $A[i]$ and pivot

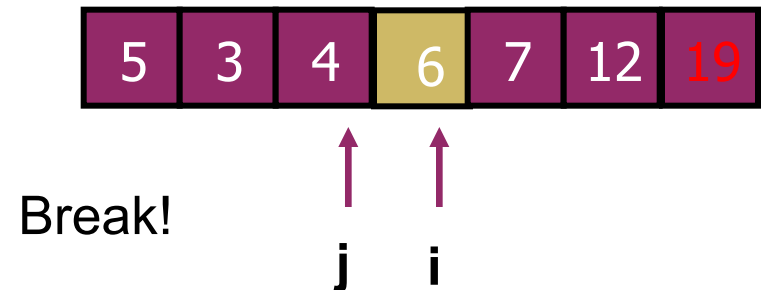
□ Result:

□ $A[k] \leq \text{pivot}$, for $k < i$

□ $A[k] \geq \text{pivot}$, for $k > i$



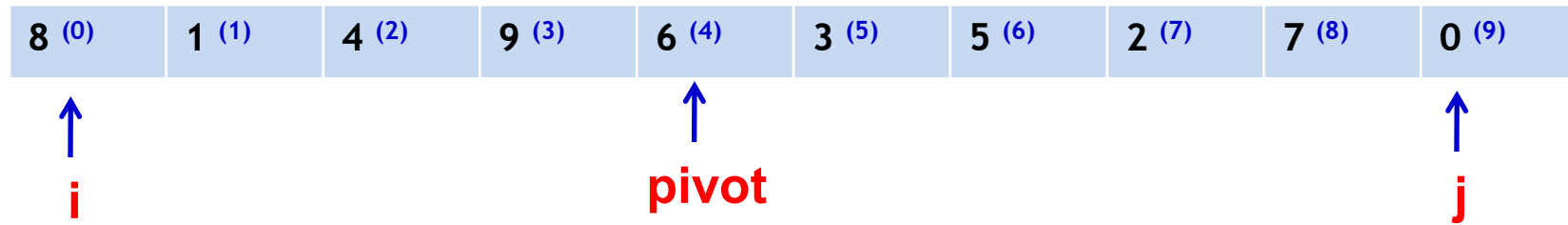
swap $A[i]$ and pivot



DETAILED VIEW

QuickSort :Partitioning strategy

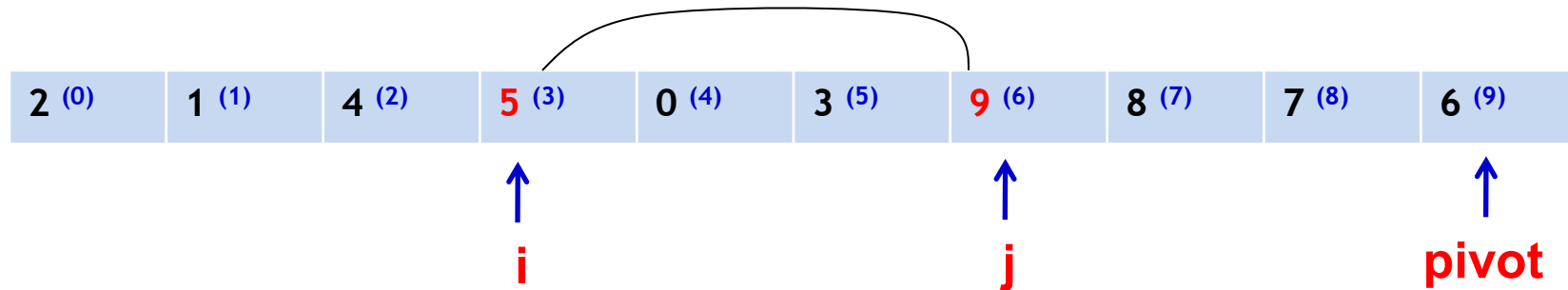
Example



- ⦿ The basic algorithm
- ⦿ While *i* is to the left of *j*,
 - we move *i* right, skipping over elements smaller than the pivot
 - We move *j* left, skipping over elements larger than the pivot
- ⦿ When *i* and *j* have stopped,
 - *i* is pointing at a large element, and
 - *j* is pointing at a small element
- ⦿ If *i* is to the left of *j*,
 - those elements are swapped
- ⦿ The effect is to push a large element to the right and a small element to the left.
- ⦿ In the example above, *i* would not move and *j* would slide over one place

QuickSort :Partitioning strategy

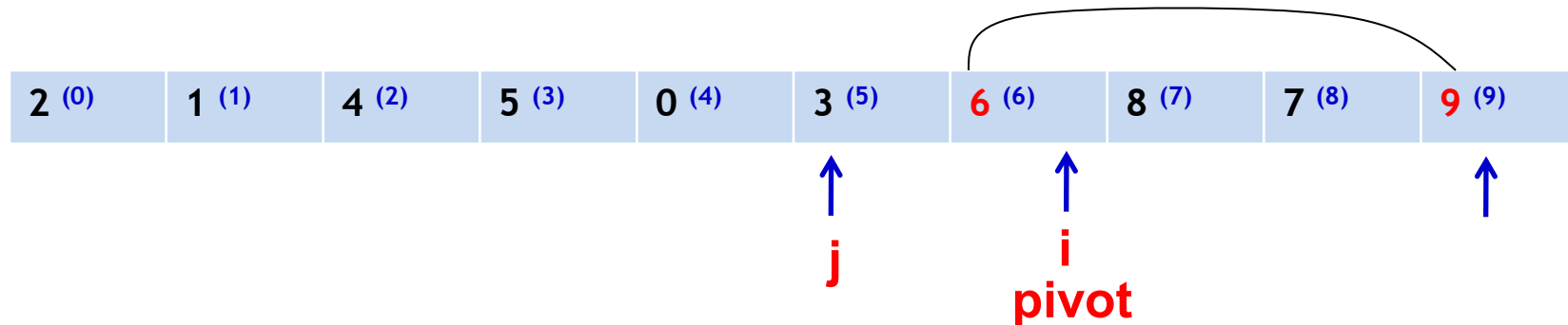
Step 2



- ⦿ $A[i] = 2 < \text{pivot}$
 - Move Right
- ⦿ $A[i] = 1 < \text{pivot}$
 - Move Right
- ⦿ $A[i] = 4 < \text{pivot}$
 - Move Right
- ⦿ $A[i] = 9 > \text{pivot}$
 - Stop i right over here
- ⦿ $A[j] = 8 > \text{pivot}$
 - Move Left
- ⦿ $A[j] = 5 < \text{pivot}$
 - Stop j right over here
- ⦿ **Swap $A[i]$ and $A[j]$**

QuickSort :Partitioning strategy

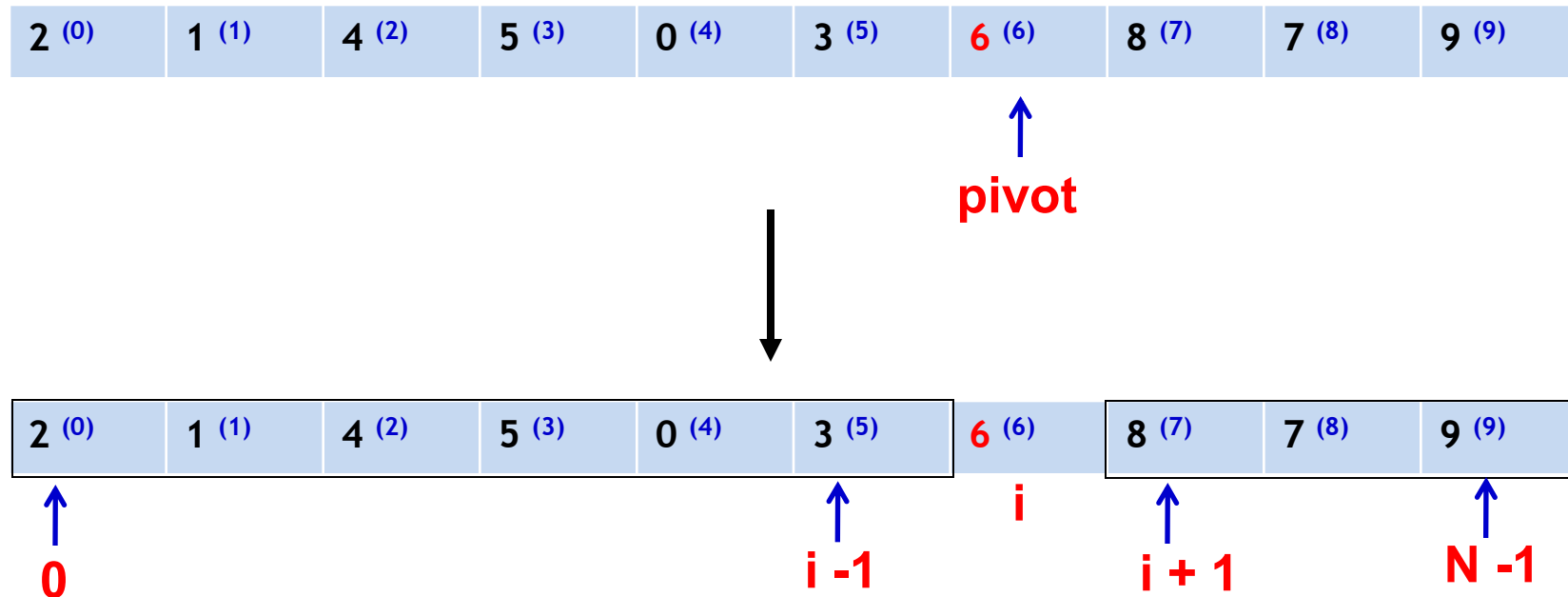
Step 3



- ⦿ $A[j] = 5 < \text{pivot}$
 - Move Right
- ⦿ $A[j] = 0 < \text{pivot}$
 - Move Right
- ⦿ $A[j] = 3 < \text{pivot}$
 - Move Right
- ⦿ $A[j] = 9 > \text{pivot}$
 - Stop i right over here
- ⦿ $A[j] = 9 > \text{pivot}$
 - Move Left
- ⦿ $A[j] = 3 < \text{pivot}$
 - Stop j right over here
- ⦿ i and j have crossed
 - So no swap for $A[i]$ and $A[j]$
- ⦿ Instead Swap $A[i]$ and $A[\text{pivot}]$

QuickSort(α , left, $i-1$);
QuickSort(α , $i+1$, right);

QuickSort : Recursive calls



PICKING THE PIVOT



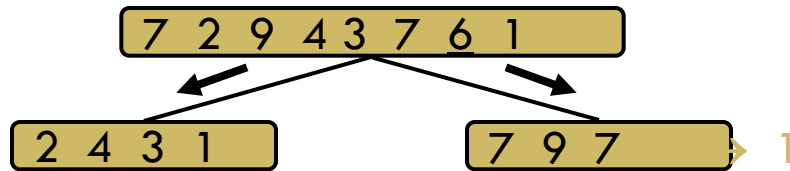
Picking the Pivot

37

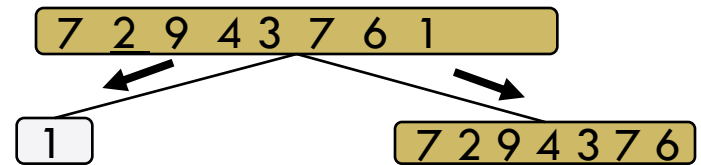
- There are several ways to pick a pivot.
- Objective:
 - ▣ Choose a pivot so that we will get 2 partitions of (almost) equal size.

Picking the Pivot

- Consider a recursive call of quick-sort on a sequence of size s
 - ▣ **Good call:** the sizes of L and G are each less than $3s/4$
 - ▣ **Bad call:** one of L and G has size greater than $3s/4$

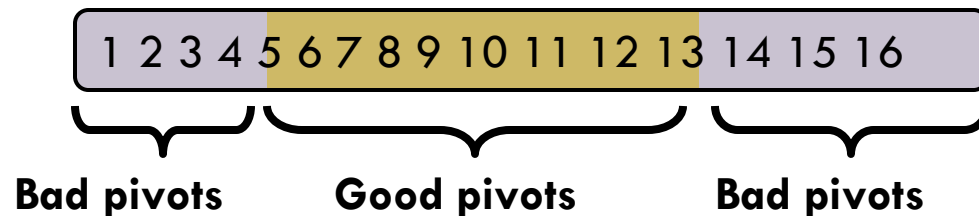


Good call



Bad call

- A call is good with probability $1/2$
 - ▣ $1/2$ of the possible pivots cause good calls:



Picking the Pivot

39

□ Use the first element as pivot

- if the input is random, ok.
- if the input is presorted (or in reverse order)
 - all the elements go into S_2 (or S_1).
 - this happens consistently throughout the recursive calls.
 - results in $O(N^2)$ behavior.

Picking the Pivot

40

Use the first element as pivot

1- The popular, uninformed choice:

- Use the first element as the pivot
 - ▣ This is acceptable if the input is random
 - ▣ But, if the input is presorted or in reverse order, then the pivot provides a poor partition, because virtually all the elements go into S_1 or S_2
 - ▣ It happens consistently throughout the recursive calls
- ➔ Quicksort will take quadratic time to do essentially nothing at all, which is quite embarrassing

Picking the Pivot

41

□ Choose the pivot randomly

2- A Safe Maneuver

- A safe course is merely to choose the pivot randomly
 - ▣ This strategy is generally perfectly safe, unless the random number generator has a flaw
 - ▣ generally safe,
 - ▣ However, random number generation is generally an expensive commodity and does not reduce the average running time of the rest of the algorithm at all

Picking the Pivot

42

- **Use the median of the array (ideal pivot)**
- The best choice of pivot would be the median of the file.
 - ▣ The median of a group of n numbers is the $(n/2)$ -th largest number
 - ▣ Unfortunately, this is hard to calculate and would slow down quicksort considerably
 - ▣ Partitioning always cuts the array into roughly half
 - ▣ However, hard to find the exact median
- A good estimate can be obtained by picking three elements randomly and using the median of these three as pivot.
 - ▣ The randomness turns out not to help much
- So, the common course is to use as pivot the median of the left, right and center elements

Picking the Pivot

43

- Use the median of the array (ideal pivot)
- So, the common course is to use as pivot the median of the left, right and center elements

$A = \{8, 1, 4, 9, 6, 3, 5, 2, 7, 0\} \longrightarrow \text{center} = A[(\text{left} + \text{right})/2]$
 $= A[(0+8)/2] = A[4] = 6$
pivot: $v = \text{median}(8, 6, 0) = 6$

Picking the Pivot

44

□ Median-of-three partitioning

- eliminates the bad case for sorted input.
- reduces the number of comparisons by 14%.

□ Example:

- Compare just three elements: the leftmost, rightmost and center
- Swap these elements if necessary so that
 - $A[\text{left}] = \text{Smallest}$
 - $A[\text{right}] = \text{Largest}$
 - $A[\text{center}] = \text{Median of three}$

Picking the Pivot

45

□ Example:

- Compare just three elements: the leftmost, rightmost and center
- Swap these elements if necessary so that
 - $A[\text{left}] = \text{Smallest}$
 - $A[\text{right}] = \text{Largest}$
 - $A[\text{center}] = \text{Median of three}$
- Pick $A[\text{center}]$ as the pivot.
- Swap $A[\text{center}]$ and $A[\text{right} - 1]$ so that the pivot is at the second last position, if $A[\text{right}]$ is the largest.

Picking the Pivot

46

□ Example:

```
int center = ( left + right ) / 2;  
if( a[ center ] < a[ left ] )  
    swap( a[ left ], a[ center ] );  
if( a[ right ] < a[ left ] )  
    swap( a[ left ], a[ right ] );  
if( a[ right ] < a[ center ] )  
    swap( a[ center ], a[ right ] );  
  
    // Place pivot at position right - 1  
swap( a[ center ], a[ right - 1 ] );
```

We only need to partition A[**left + 1**, ..., **right - 2**].

Median-of-three Partitioning

47



$A[\text{left}] = 2$, $A[\text{center}] = 13$,
 $A[\text{right}] = 6$



Swap $A[\text{center}]$ and $A[\text{right}]$



Choose $A[\text{center}]$ as **pivot**

↑
pivot

We only need to partition $A[\text{left} + 1, \dots, \text{right} - 2]$.



Swap pivot and $A[\text{right} - 1]$

↑
pivot

Median-of-three Partitioning

48

- Recursive case: QuickSort(a, left, right)
 pivot = median3(a, left, right);
 Partition a[left ... right] into a[left ... i-1], i, a[i+1 ... right];
 QuickSort(a, left, i-1);
 QuickSort(a, i+1, right);
- Base case: when do we stop the recursion?
 - ▣ when left $\geq$ right.

QuickSort: Core Function

```
void q_sort( input_type a[], int left, int right ) {  
    int i, j; int pivot;  
    if( left + CUTOFF <= right )    {  
        pivot = median3( a, left, right );  
        i = left; j = right - 1;  
        for ( ; ; )    {  
            while( a[++i] < pivot );  
            while( a[--j] > pivot );  
            if( i < j )  
                swap( &a[i], &a[j] );  
            else  
                break;  
        } //end for loop  
        swap( &a[i], &a[right-1] ); /*restore pivot*/  
        q_sort( a, left, i - 1 ); // left recursive call  
        q_sort( a, i + 1, right ); // right recursive call  
    }  
}
```

QuickSort: Medians

```
/* Return median of left, center, and right. */  
/* Order these and hide pivot */  
int median3( input_type a[], int left, int right )  
{  
    int center;  
    center = (left + right) / 2;  
    if ( a[left] > a[center] )  
        swap( &a[left], &a[center] );  
    if ( a[left] > a[right] )  
        swap( &a[left], &a[right] );  
    if ( a[center] > a[right] )  
        swap( &a[center], &a[right] );  
    /* a[left] <= a[center] <= a[right] */  
    swap( &a[center], &a[right-1] );  
    return a[right-1]; /* return pivot */  
}
```

QuickSort: Medians

```
/* Return median of left, center, and right. */
```

```
/* Order these and hide pivot */
```

int	8 (0)	1 (1)	4 (2)	9 (3)	6 (4)	3 (5)	5 (6)	2 (7)	7 (8)	0 (9)
-----	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------

```
{
int center;
```

```
center = left / 2;
```

```
if ( a[left] > a[center] )
```

```
    swap( &a[left], &a[center] );
```

8 > 6

```
if ( a[left] > a[right] )
```

```
    swap( &a[left], &a[right] );
```

6 > 0

```
if ( a[center] > a[right] )
```

```
    swap( &a[center], &a[right] );
```

8 > 6

```
/* a[left] <= a[center] <= a[right] */
```

```
swap( &a[center], &a[right-1] );
```

```
return a[right-1]; /* return pivot */
```

```
}
```

QuickSort: Core Function

```
void q_sort( input_type a[], int left, int right ) {
```

int	0 (0)	1 (1)	4 (2)	9 (3)	7 (4)	3 (5)	5 (6)	2 (7)	6 (8)	8 (9)
-----	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------

```
if ( left < right ) {  
    ↑  
    { i = left  
      ↑  
      pivot = median3( a, left, right );  
      center  
      i = left; j = right - 1;
```

```
    for ( ; ; )
```

```
        {
```

```
            while ( a[++i] < pivot );
```

```
            while ( a[--j] > pivot );
```

```
            if ( i < j )
```

```
                swap( &a[i], &a[j] );
```

```
            else
```

```
                break;
```

```
        } //end for loop
```

```
    swap( &a[i], &a[right-1] );
```

```
    q_sort( a, left, i-1 );
```

```
    q_sort( a, i+1, right );
```

```
}
```

```
}
```

```
/* while i is to the left of j,
```

- move **i** **right**, skipping over elements **smaller** than the **pivot**

- move **j** **left**, skipping over elements **larger** than the **pivot**. */

```
/* If i is to the left of j,
```

- those elements are swapped */

```
    j = right - 1  
    ↑  
    pivot
```

QuickSort: Core Function

```
void q_sort( input_type a[], int left, int right ){
```



```
if( left < right )
{
    left
    pivot = median3( a, left, right );
```

i

i

right - 1
pivot

```
    i = left; j = right - 1;
    for ( ; ; )
    {
        while ( a[++i] < pivot );
        while ( a[--j] > pivot );
        if ( i < j )
            swap( &a[i], &a[j] );
        else
            break;
    } //end for loop
```

```
    swap( &a[i], &a[right-1] );
```

```
    q_sort( a, left, i - 1 );
```

```
    q_sort( a, i + 1, right );
```

```
}
```

/* loop terminated when (i > j) i.e., i and j have crossed → so no swap is performed */

/*restore pivot*/

// left recursive call

// right recursive call

```
}
```

Median-of-three Partitioning

54

```
if( left + 10 <= right )  
{
```

```
    Comparable pivot = median3( a, left, right );
```

Choose pivot

```
        // Begin partitioning
```

```
        int i = left, j = right - 1;  
        for( ; ; )  
        {  
            while( a[ ++i ] < pivot ) { }  
            while( pivot < a[ --j ] ) { }  
            if( i < j )  
                swap( a[ i ], a[ j ] );  
            else  
                break;  
        }
```

Partitioning

```
        swap( a[ i ], a[ right - 1 ] ); // Restore pivot
```

```
        quicksort( a, left, i - 1 );    // Sort small elements  
        quicksort( a, i + 1, right );  // Sort large elements
```

Recursion

```
    }  
    else // Do an insertion sort on the subarray  
        insertionSort( a, left, right );
```

For small arrays

Median-of-three Partitioning

55

- The partitioning code we just saw works only if pivot is picked as **median-of-three**.

- ▣ $A[\text{left}] \leq \text{pivot}$ and $A[\text{right}] \geq \text{pivot}$
- ▣ Need to partition only
 $A[\text{left} + 1, \dots, \text{right} - 2]$

- j will not run past the beginning
 - ▣ because $A[\text{left}] \leq \text{pivot}$

- i will not run past the end
 - ▣ because $A[\text{right}-1] = \text{pivot}$

```
int i = left, j = right - 1;
for( ; ; )
{
    while( a[ ++i ] < pivot ) { }
    while( pivot < a[ --j ] ) { }
    if( i < j )
        swap( a[ i ], a[ j ] );
    else
        break;
}
```

Quick Sort vs Merge Sort

56

- Both quick sort and merge sort take $O(N \log N)$ in the average case.
- But quick sort is faster in the average case:
 - ▣ The inner loop consists of an increment/decrement (by 1, which is fast), a test and a jump.
 - ▣ There is no extra juggling as in merge sort.

```
int i = left, j = right - 1;
for( ; ; )
{
    while( a[ ++i ] < pivot ) { }
    while( pivot < a[ --j ] ) { }
    if( i < j )
        swap( a[ i ], a[ j ] );
    else
        break;
}
```

inner loop

QUICK SORT TIME COMPLEXITY



Quick Sort

58

- **Running time**
 - ▣ pivot selection: constant time, i.e. $O(1)$
 - ▣ partitioning: linear time, i.e. $O(N)$
 - ▣ running time of the two recursive calls

- $T(N) = T(i) + T(N - i - 1) + cN$
 - ▣ i : number of elements in S_1
 - ▣ c is a constant

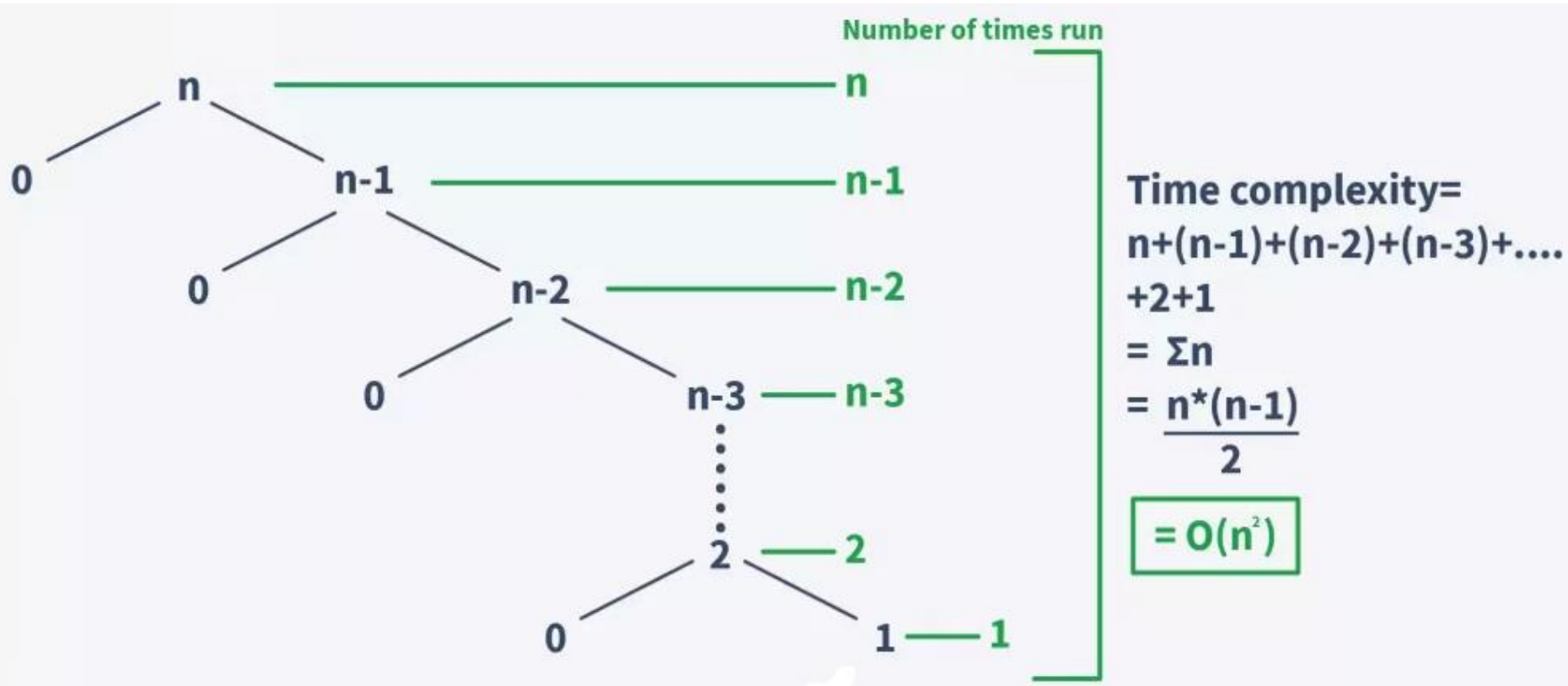
Quick Sort ... Worst Case

59

- The Worst-case occurs **when either of the two partitions is unbalanced.**
 - ▣ This generally happens when the **greatest or smallest element is selected as the pivot.**
- In this case, the pivot lies at the extreme of the array, and one subarray is always empty while the other contains **$n - 1$** elements.
 - ▣ This way, in the first call, the array is divided into two subarrays of 0 & $n - 1$ elements respectively. For the second call, the array with $n - 1$ elements is divided into two subarrays of 0 & $n - 2$ elements respectively.

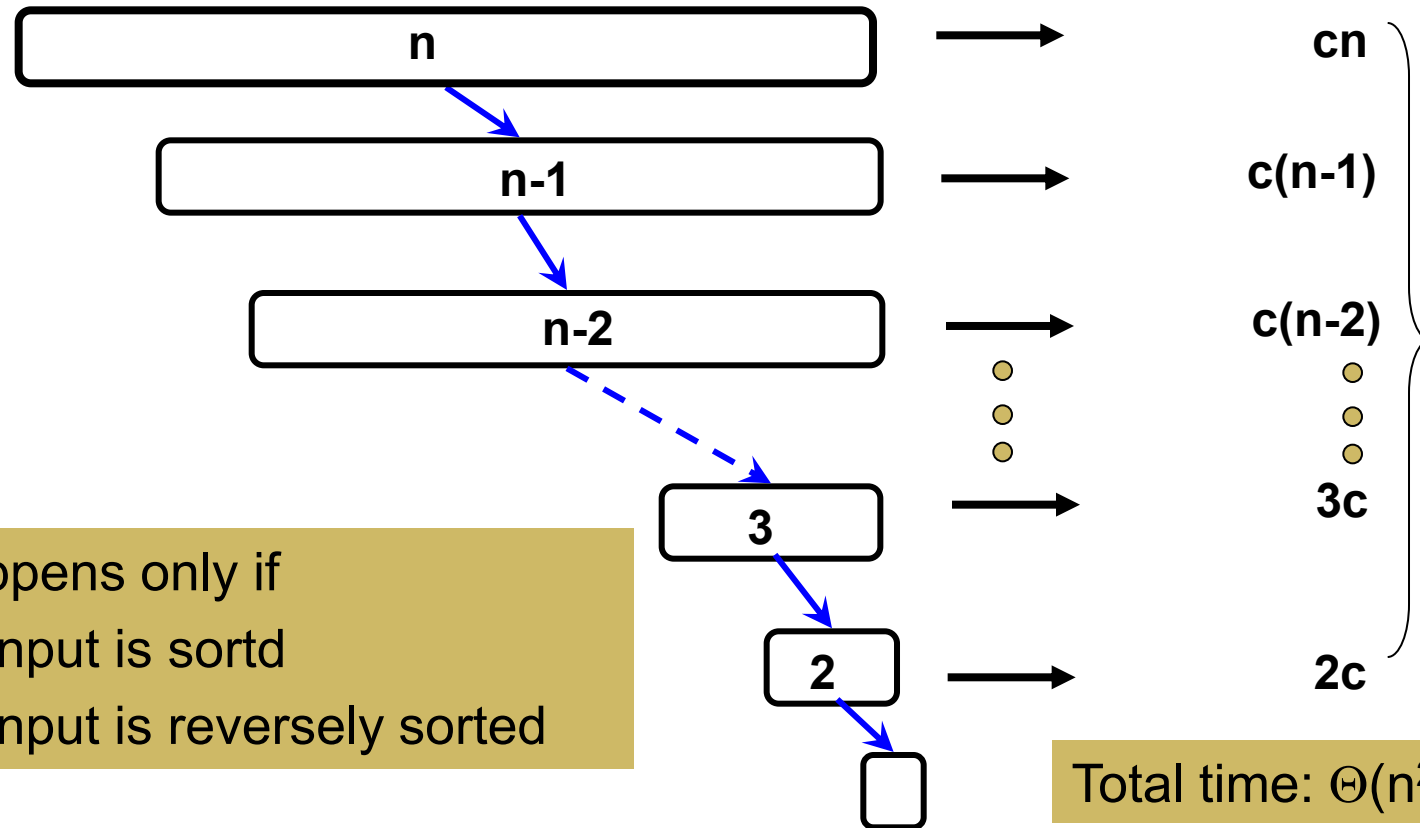
Quick Sort ... Worst Case

60



Quick-Sort: Worst Case

- Unbalanced Partition



Happens only if

- input is sorted
- input is reversely sorted

Quick Sort ... Worst Case

62

□ What will be the worst case?

- ▣ The pivot is the smallest element, all the time
- ▣ Partition is always unbalanced

$$T(N) = T(N - 1) + cN$$

$$T(N - 1) = T(N - 2) + c(N - 1)$$

$$T(N - 2) = T(N - 3) + c(N - 2)$$

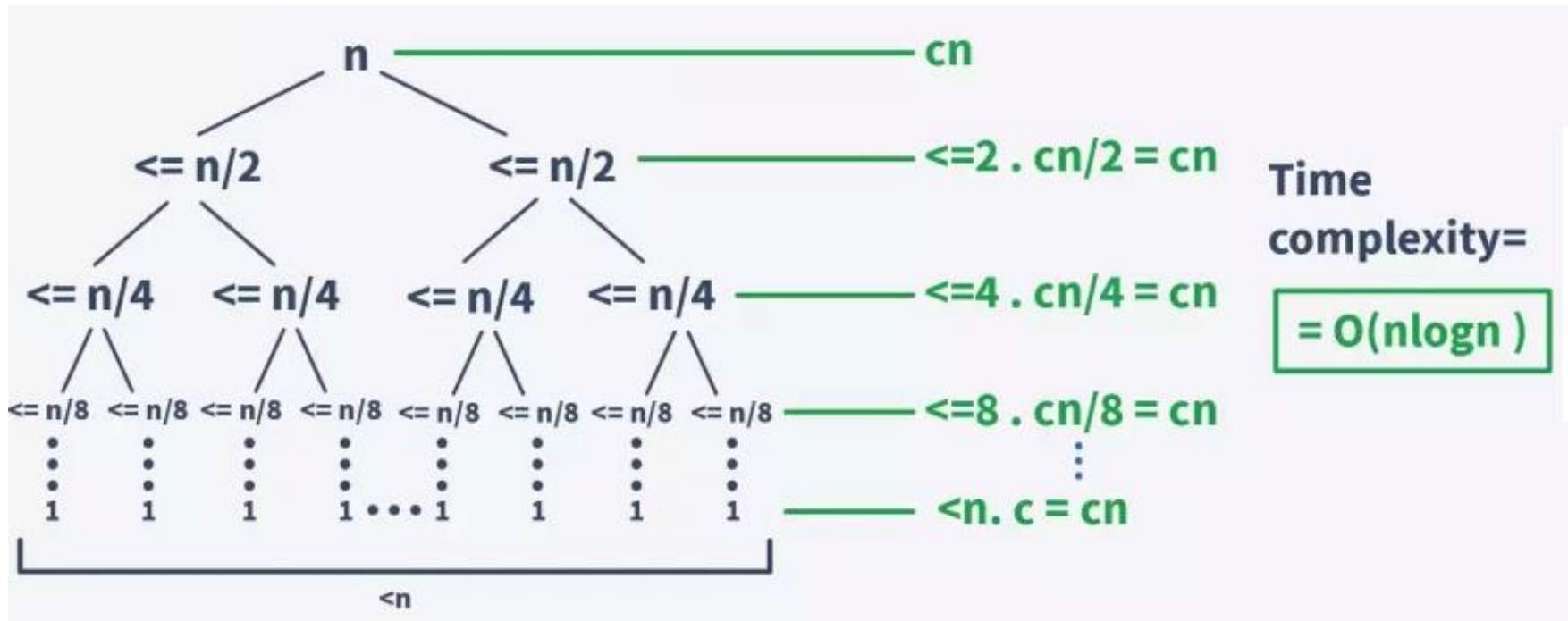
⋮

$$T(2) = T(1) + c(2)$$

$$T(N) = T(1) + c \sum_{i=2}^N i = O(N^2)$$

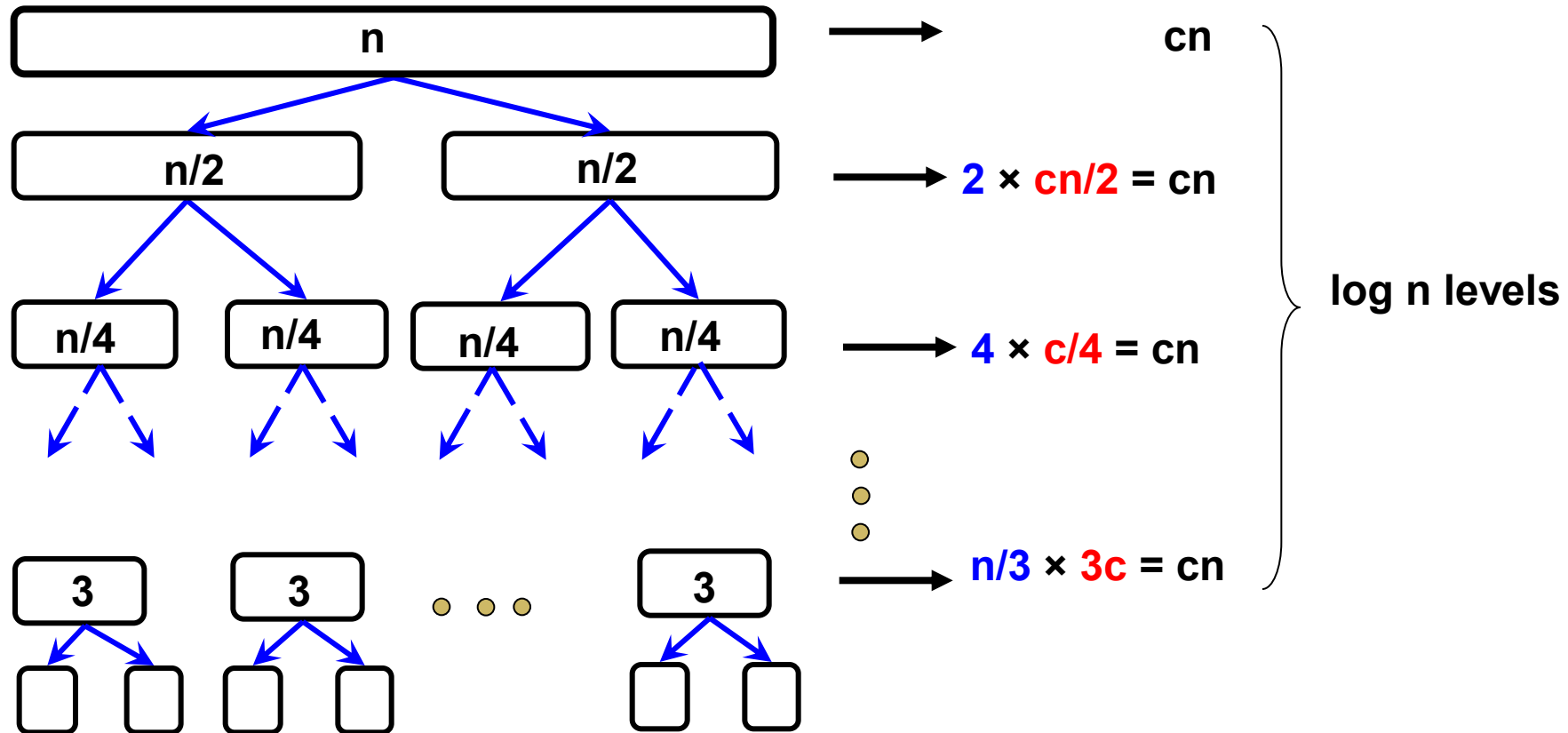
Quick Sort ... Best Case

63



Quick-Sort: Best Case

- Even Partition



Total time: $\Theta(n \log n)$

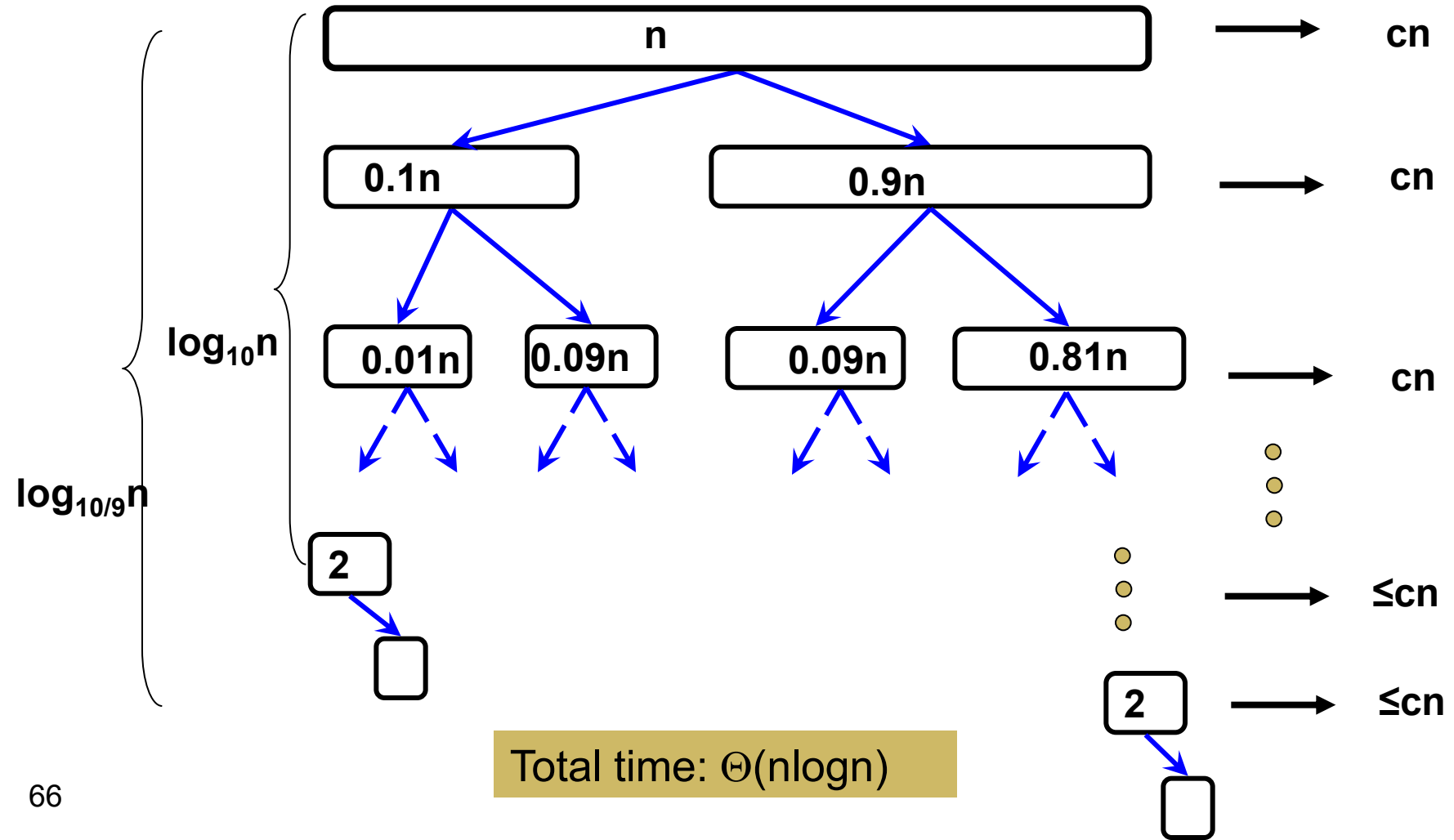
Quick Sort ... Best Case

65

- **What will be the best case?**
 - ▣ Partition is perfectly balanced.
 - ▣ Pivot is always in the middle (median of the array).
- $T(N) = T(N/2) + T(N/2) + cN = 2T(N/2) + cN$
- This recurrence is similar to the merge sort recurrence.
- The result is $O(N\log N)$.

Quick-Sort: an Average Case

- Suppose the split is $1/10 : 9/10$



Quick Sort

67

Summary

□ Time

- Most of the work done in partitioning.
- Average case takes $\Theta(n \log(n))$ time.
- Worst case takes $\Theta(n^2)$ time

□ Space

- Sorts in-place, i.e., does not require additional space

Quick Sort

68

- Divide and Conquer
- **Merge-Sort**
 - ▣ Most of the work done in Merging
 - ▣ $\Theta(n \log(n))$ time
 - ▣ $\Theta(n)$ space
- **Quick-Sort**
 - ▣ Most of the work done in partitioning
 - ▣ Average case takes $\Theta(n \log(n))$ time
 - ▣ Worst case takes $\Theta(n^2)$ time
 - ▣ $\Theta(1)$ space

Acknowledgement

69

Introduction to Algorithms

Thomas H. Cormen, Charles E. Leiserson,
Ronald L. Rivest, and Clifford Stein